`Source[Int, NotUsed]` expresses that this producer generates integers, while `NotUsed` takes the position of the materialized value. To understand what *materialized* means, you need more context than the `Source` alone. You learn about `NotUsed` in section 10.1.7.

10.1.4 Flow

The transformation function—also called the *processor*—is the filtering part of the previous stream:

```
val processor: Flow[Int, Int, NotUsed] = Flow[Int].filter(_ % 2 == 0)
```

The first `Int` in `Flow[Int, Int, NotUsed]` indicates that the flow accepts integers as input and the second produces integers as output. `NotUsed`, again, refers to the materialization of the flow.

10.1.5 Sink

The third element is the consumer:

```
val consumer: Sink[Int, Future[Done]] =  Sink.foreach(i => storeDB(i))
```

`Sink[Int, Future[Done]]` indicates that it consumes integers and produces a `Future[Done]`. This byproduct is called a *materialized value* and is produced when the stream is executed. It is the output of the graph. This can be confusing at first because the output of the graph in the future will be `Done`, while the values it processes are integers.

10.1.6 Blueprint

The blueprint is the product of mixing and matching the different stages. It's an abstraction that gives you composability and testability:

```
val blueprint
    : RunnableGraph[scala.concurrent.Future[akka.Done]] =
  producer.via(processor).toMat(consumer)(Keep.right)
```

Once stages compose a `RunnableGraph`, you have a function that is decoupled from the execution of the stream. You can pass it around and execute it at will:

```
val future: Future[Done] = blueprint.run
```

10.1.7 Materialization

Materialization is the execution of a blueprint, and its by-products are called *materialized values*. Materialization is similar to an `ActorRef` in that one behavior spawns multiple `ActorRef`s. With Akka Streams, you have one graph; each time you run it, you get its instance: a tuple with materialized values.